Azure Spot VM ML Workload Guide

Operational Runbook for Interruptible ML Training on Standard_E16ps_v5 (Arm64)

This document details the operational execution steps to submit, run, and auto-resume an interruptible machine learning training job on a bare-metal standalone Azure Spot VM using decentralized Azure Blob Storage for state permanence.

Step 1: Establish Secure Connection

Once your Terraform deployment successfully provisions the infrastructure, retrieve the public IP address of the newly allocated instance and establish an SSH connection:

```
ssh azureuser@<YOUR_VM_PUBLIC_IP>
```

Step 2: Authenticate & Mount Persistent Blob Storage

Because the infrastructure attaches a User-Assigned Managed Identity natively via RBAC (Storage Blob Data Contributor role), there is no need to hardcode or pass storage credentials inside the environment. Utilize **BlobFuse2** to seamlessly mount the remote blob architecture into a native Linux directory pathway.

1. Create a local directory path acting as your mount endpoint:

   ```
   mkdir -p ~/mnt_artifacts
   ```

2. Configure the BlobFuse2 `config.yaml` environment mapping profile:

```
logging:
  type: syslog
  level: log_term

components:
  - libfuse
  - file_cache
  - attr_cache
  - azstorage

libfuse:
  attribute-default-permissions: "true"

file_cache:
  path: /tmp/blobfuse2-cache
  timeout-sec: 120
  max-size-mb: 4096

azstorage:
  type: block
  account-name: stmlspotminimal123
  container: artifacts
  mode: identity
  identity-client-id: <MANAGED_IDENTITY_CLIENT_ID>
```

3. Mount the container using the definition file:

```
blobfuse2 mount ~/mnt_artifacts --config-file=./config.yaml
```

Step 3: Implement Checkpoint Logic in Training Script

To successfully recover from arbitrary evictions, your execution script (e.g., `train.py`) must dynamically check for pre-existing checkpoint states inside the mounted storage volume before initializing standard baseline weights.

```
import os
import torch

# Define persistent checkpoint mount target path
checkpoint_path = "/home/azureuser/mnt_artifacts/checkpoint.pt"

# Verify state history for automatic recovery
if os.path.exists(checkpoint_path):
    print("Found existing checkpoint! Resuming training...")
    checkpoint = torch.load(checkpoint_path)
    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    start_epoch = checkpoint['epoch'] + 1
else:
    print("No checkpoint found. Initializing training from scratch.")
    start_epoch = 0

# Engine execution loop
for epoch in range(start_epoch, total_epochs):
    train_one_epoch(model, optimizer, data_loader)

    # Snapshot frequently to limit data delta risk on sudden eviction
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
    }, checkpoint_path)
```

Step 4: Execute the Non-Interactive Job Process

To ensure session continuity independent of your local terminal or connection reliability, dispatch the long-running execution thread in the background using isolated background utilities:

```
nohup python3 train.py > training.log 2>&1 &
```

Alternatively, manage interactive processes inside an isolated multiplexer window:

```
tmux new -s ml_session
python3 train.py
# Press Ctrl+B then D to safely detach
```

Handling Evictions:
When Azure reclaims capacity, this standalone instance terminates immediately. Because the `eviction_policy` is explicitly designated as `Delete`, the compute engine and OS footprint are completely cleaned to ensure zero passive run costs. Simply reissue `terraform apply` at a later window to create a clean identical environment; the new instance mounts the exact same storage block and securely continues from the last successfully saved epoch checkpoint.